

Evaluating regression model performance: MSE, RMSE, and R-squared

1. What You'll Learn in This Section

In this lesson, you'll learn to:

- Calculate mean square error (MSE), root mean square error (RMSE), and R-squared (R^2) to measure a regression model's performance by hand.

- Explain why MSE is preferred over mean absolute error (MAE) for training regression models.

- Interpret an R^2 value, including what a value near 1, near 0, or negative actually means about model performance.

- Apply the correct evaluation metric when comparing models' performance on the same dataset versus across different datasets.

2. Detailed Explanation

Why regression models need evaluation metrics

Imagine two teams each build a regression model to fit the same five training data points. Team A's model is drawn as an orange line through the points; Team B's model is drawn as a green line, with white dots marking the actual data. Both are simple toy examples with only five points, since more than one input feature cannot easily be visualized alongside the output on a flat, two-dimensional plot. A real dataset would typically have thousands of points and more than one feature.

The question is: which model is more accurate, and which one should be handed to a customer? Just by eyeballing the two plots, it's hard to say for certain. One line might look like it passes closer to the points, but a visual impression is not convincing evidence. This is exactly the problem evaluation metrics solve. Instead of relying on a gut feeling, you compute metrics like MSE and R^2 and use them as objective evidence of which model genuinely performs better.

Residuals and mean square error (MSE)

Given a set of input values (X) and their actual output values (Y), a regression model produces a function $f(x)$ that predicts a value $\hat{Y}$ ("Y-hat") for each X . The residual is the gap between what actually happened and what the model predicted:

$$\text{residual} = Y - Y_{\text{hat}}$$

Mean square error (MSE) squares each residual, sums the squared residuals across every data point, and divides by the total number of points N:

$$\text{MSE} = (1/N) * \sum (Y_i - \hat{Y}_i)^2, \text{ for } i = 1 \text{ to } N$$

In plain words: MSE is the average of the squared mistakes, where a "mistake" is the gap between the actual and predicted value at each point.

Three metrics share this same residual idea: MSE, mean absolute error (MAE), and root mean square error (RMSE). MSE is the preferred choice for training and evaluating regression models, for two reasons:

1. It emphasizes large errors. Squaring a residual makes big mistakes stand out far more than small ones. A residual of -2 becomes 4 when squared, but a residual of +5 becomes 25 — more than six times larger, not just 2.5 times larger. During training, this is desirable: a model that makes a large error should be penalized more heavily so it improves faster in later training iterations. MAE, by contrast, would score a residual of +5 as simply 5, with no extra amplification relative to smaller errors.
2. It's easier to differentiate. Squaring a term produces a simpler derivative than other formulations — the derivative of x^2 with respect to x is just $2x$. This simplicity helps machine learning algorithms find optimal parameter values faster, which is why MSE, not MAE, is the standard choice for training models.

RMSE is not a fundamentally different metric — it's simply the square root of MSE, so computing RMSE still means computing MSE first.

Worked example: predicting apartment prices

A running example predicts apartment prices (in lakhs of rupees) in a Bangalore society from house size (in square feet), using five houses:

House	Size (sq ft)	Actual price (lakhs)	Predicted price (lakhs)	Residual (Actual – Predicted)	Squared residual
1	600	40	40	0	0
2	800	55	50	5	25
3	1000	58	60	-2	4
4	1200	65	70	-5	25

5	1500	90	85	5	25
---	------	----	----	---	----

Sum of squared residuals = $0 + 25 + 4 + 25 + 25 = 79$

$$\text{MSE} = 79 / 5 = 15.8$$

Picture this visually: white circles mark the actual data points, and a yellow line with yellow squares marks the model's predicted function and predicted points. Dashed red lines connect each actual point to its predicted counterpart. Each dashed line's length is that point's residual. Geometrically, MSE is the accumulated (squared) length of all those dashed lines, normalized by the number of points.

Properties of MSE:

It's an error score, so lower is always better — there's no situation where a higher MSE is desirable. Because residuals are squared, MSE is always non-negative. An MSE of exactly zero means every prediction was perfectly accurate. This is unrealistic in practice; a model is shipped with some small, non-zero error, and how much error is "acceptable" depends on the domain — there's no universal fixed threshold.

RMSE: reporting error to a customer

Because MSE squares the error, its numeric value overstates the real-world error in a way that can mislead a non-technical audience. An MSE of 15.8 (lakhs-squared, from the worked example) is not directly interpretable as "15.8 lakhs of error." A customer hearing that number might reasonably — but incorrectly — assume the true price could be off by roughly 15.8 lakhs, when the real, unsquared discrepancy is much smaller.

RMSE, the square root of MSE, restores the value to its original unit scale:

$$\text{RMSE} = \sqrt{15.8} \approx 3.97 \text{ (rounded to two decimal places)}$$

The practical convention: MSE is used internally, during training and testing, to optimize and evaluate the model. RMSE is what gets reported externally, to a customer or stakeholder, because it reflects the actual expected error rather than an inflated squared number. If a model predicts a price of 50 lakhs and the RMSE is about 4 lakhs, the customer correctly understands the real price could realistically be off by roughly ± 4 lakhs. That is a far more useful figure than the much larger MSE value.

Outlier sensitivity of MSE

As a measure of model performance, MSE has one significant drawback: it's highly sensitive to outliers — data points far from the general pattern. For example, an apartment that's small but priced very high because of an unusually good interior is an outlier. If such a point is included during training, it can distort the overall fitted function (the "hypothesis"). The model gets pulled away from a good fit on the majority of ordinary houses just to accommodate the one unusual point.

Watch out for: the fix is to identify and remove outliers before training, not to abandon MSE. It's acceptable if the resulting model predicts an outlier poorly, as long as it stays reliable on the regular, similarly-built houses — those unusual cases can be checked manually.

There's also a rule for fairly comparing performance. Two models can only be fairly compared on MSE if they were both trained on the same dataset, tested on the same dataset, and use the same algorithm. Comparing an MSE computed on one dataset against an MSE computed on a different dataset is not a valid comparison.

R-squared (R^2): the baseline model idea

R^2 measures a regression model's performance relative to a baseline model — the simplest possible prediction strategy, with no analytical or mathematical reasoning applied at all. Given a set of X and Y values, the most naive reasonable guess for a new prediction, without doing any modeling, is to always predict the mean of the observed Y values. This "always predict the mean" strategy is the baseline model. It's not necessarily a bad model — it's simply the best possible guess without any statistical technique. If the actual Y values don't vary much from their mean, this naive guess isn't a poor predictor at all.

The baseline model's error is the difference between each actual Y and the mean of Y, squared and summed. A properly built linear regression model, which does apply mathematical and statistical reasoning, is expected to produce a much smaller error than this naive baseline.

R^2 formula and interpretation

```
SS_residual = Σ (Y_i - Y_hat_i)^2      (the model's actual squared error – same numerator
idea as MSE)
SS_total     = Σ (Y_i - Y_mean)^2      (the baseline model's squared error, against the
mean)

R^2 = 1 - (SS_residual / SS_total)
```

If the model performs very well, SS_{residual} is small relative to SS_{total} , the fraction is close to 0, and R^2 is close to 1.

If the model performs exactly as well as the baseline, SS_{residual} equals SS_{total} , the fraction is 1, and R^2 is 0.

If the model performs worse than the baseline, SS_{residual} exceeds SS_{total} , the fraction exceeds 1, and R^2 becomes negative.

R^2 ranges from 0 to 1 for models at least as good as the baseline, though it can go negative for models worse than the baseline. A value close to 1 indicates a near-perfect fit. A value close to 0 indicates the model is no better than always guessing the mean. A negative value indicates the model performs worse than the naive baseline guess.

Worked examples: good, baseline-only, and bad models

Using the same five-apartment dataset (actual prices 40, 55, 58, 65, 90):

```
Mean actual price (Y_mean) = 61.6 lakhs
```

```

SS_residual = 79          (same sum of squared residuals computed for MSE)
SS_total    = 1341       (sum of squared differences between each actual price and 61.6)

R² = 1 - (79 / 1341) ≈ 0.94 (0.9411 to four decimal places)

```

An R^2 of 0.94 reads in plain language as: the model explains about 94% of the variation in apartment prices; the remaining 6% is not captured by the model.

Two edge cases mark the boundaries of R^2 :

1. Predicting the mean for every point. If the "predicted" values are simply the mean of the actual values for every data point, the model is identical to the baseline. R^2 then evaluates to 0.00 — no improvement over the naive guess.
2. A deliberately poor, scrambled prediction. Using intentionally random, badly-chosen predicted values (illustrated with predictions like 90, 40, 85, 55, 58 against very different actual values) produces a negative R^2 , computed as approximately -2.41. This shows R^2 can fall below zero when a model performs worse than simply guessing the mean — something that wouldn't normally happen with a properly trained model, but is possible to construct artificially.

Comparing MSE and R^2 across datasets

When comparing two models, the general and safest rule is to always test them on the same dataset. This applies no matter which metric is used, and cross-dataset comparison is not recommended as a habit.

However, there's a structural difference between the two metrics. R^2 is bounded between roughly 0 and 1 (technically unbounded below zero, but with an interpretable upper bound of 1) and expresses "how much better than baseline." Because of this, it's defensible to compare R^2 values computed on two different datasets. For example, model 1 might score $R^2 = 0.9$ on one dataset and model 2 might score $R^2 = 0.8$ on a different dataset. The two scores can still be meaningfully compared, since each one independently expresses improvement over its own dataset's baseline.

MSE, by contrast, has no fixed range: it can scale to any value depending on the dataset's units and spread. Comparing an MSE value from one dataset directly against an MSE value from a different dataset is therefore not a valid comparison. When multiple datasets are involved (say, apartment data from area A, area B, and area C), the recommended reporting format is a table with models as columns and datasets as rows, or vice versa. MSE and R^2 are reported separately for each dataset, and models are compared only within the same row.

Both MSE and R^2 include a squared-error term in their formulation, so both metrics are sensitive to outliers — neither one solves the outlier problem on its own. Outliers must be explicitly identified and removed during data preparation; no evaluation metric compensates for their presence automatically.

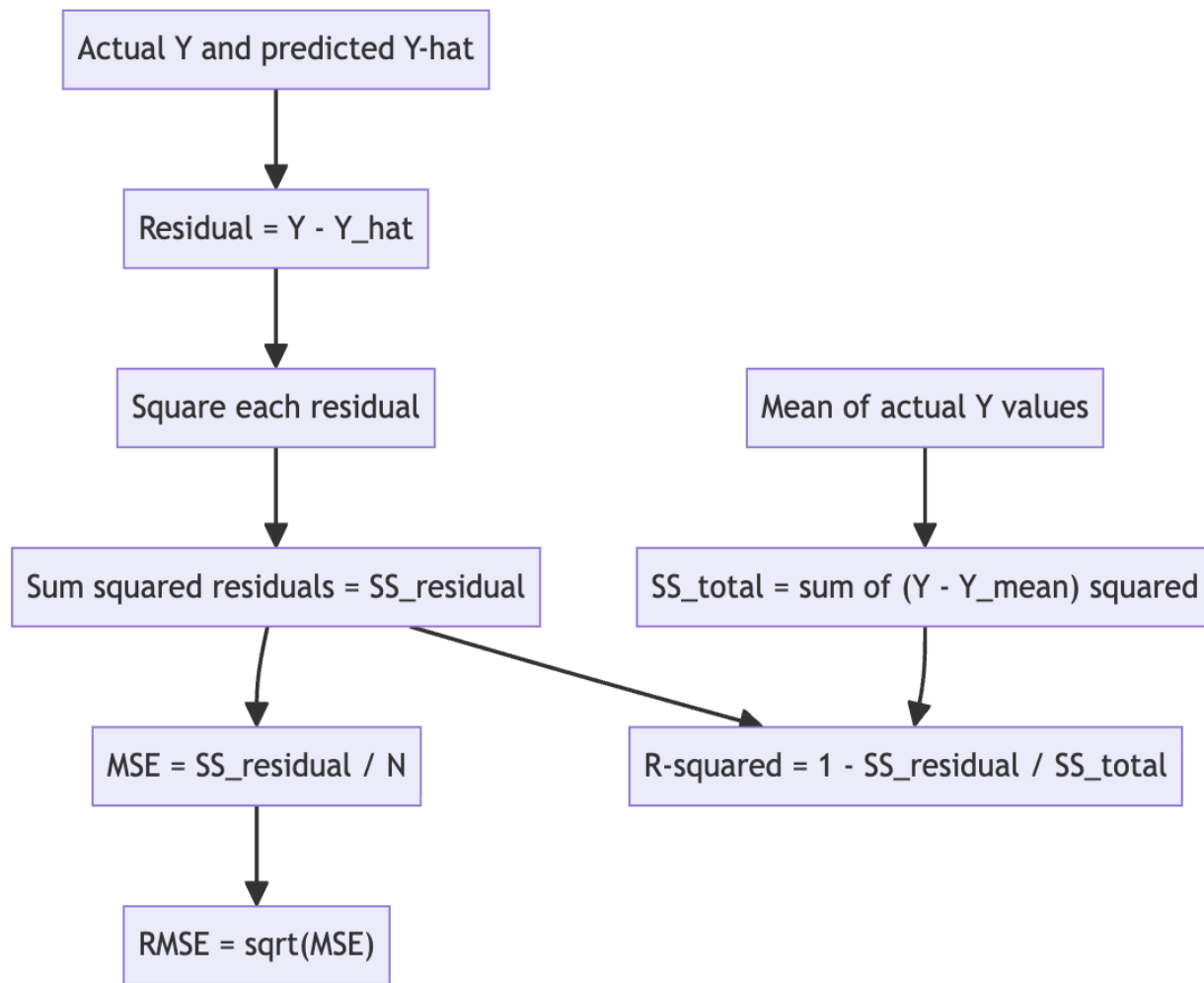
Here's how the two metrics relate to each other:

Aspect	MSE	R^2

What it measures	The real, accumulated error of the model	How much better the model is than a naive baseline (mean-only) guess
Desirable direction	Lower is better	Higher (closer to 1) is better
Range	Unbounded (any non-negative value)	0 to 1 typically; can go negative for a poor model
Cross-dataset comparison	Not valid	Defensible, though same-dataset comparison is still preferred
Outlier sensitivity	Sensitive	Sensitive
Relationship to each other	Complementary — report both; never substitute one for the other	Complementary — report both; never substitute one for the other

Both metrics share the same idea of squared residuals in their numerator. R^2 's denominator (the baseline model's error) is what makes it a genuinely different, complementary piece of information about model performance rather than a duplicate of MSE.

The diagram below traces how the same set of residuals feeds into both metrics:



Calculating MSE and R² in Python

The practical demonstration used an IPython notebook (Google Colab) to compute and visualize MSE and R², both manually and with scikit-learn, for the same five-apartment dataset.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

plt.rcParams['figure.figsize'] = (7, 4.5)
np.random.seed(42)
```

`numpy` handles the data as arrays; `pandas` stores it in a `DataFrame`.

`matplotlib.pyplot`, imported as `plt`, is called a "module" rather than a single function because it exposes many plotting functions.

Only `LinearRegression` is imported from scikit-learn's `linear_model` module, not the whole library — a pattern used throughout to import just what's needed.

`mean_squared_error` and `r2_score` are scikit-learn's built-in evaluation functions, so the formulas don't need to be hand-written. Library functions like these are treated as extremely reliable, with the chance of a wrong answer from them treated as effectively zero. `plt.rcParams['figure.figsize']` fixes every figure to a consistent size (7 inches by 4.5 inches). `np.random.seed(42)` fixes the random number generator's starting point, so anyone running the same notebook gets the exact same "random" values and output.

A pandas DataFrame is built from the same five apartments: sizes 600, 800, 1000, 1200, 1500; actual prices 40, 55, 58, 65, 90; predicted prices 40, 50, 60, 70, 85.

```
demo['residual'] = demo['actual_price'] - demo['predicted_price']
demo['squared_residual'] = demo['residual'] ** 2

n = len(demo)
sum_squared_residual = np.sum(demo['squared_residual'])
mse_manual = sum_squared_residual / n

mse_sklearn = mean_squared_error(demo['actual_price'], demo['predicted_price'])

if np.isclose(mse_manual, mse_sklearn):
    print("Manual calculation matches sklearn calculation")

rmse = round(np.sqrt(mse_manual), 2)
```

`residual` and `squared_residual` columns are added directly to the DataFrame; `len()` gives `n`, and `np.sum()` accumulates the squared residuals.

The manual computation produces sum of squared residuals = 79, `n` = 5, and MSE = 15.8 — matching the hand-worked example, and `mean_squared_error` from scikit-learn returns the same MSE.

`np.isclose()` compares the manual value against scikit-learn's, printing a confirmation only when they match.

RMSE doesn't need a separate import, since MSE is already computed manually — `np.sqrt()` is applied directly, and `round(..., 2)` formats the result. This gives RMSE ≈ 3.97 , matching the hand-worked example — the number that should be reported to a customer as the "real" error.

The same notebook plots actual vs. predicted prices: `ax.scatter(...)` draws the five actual points as white-filled circles with a blue-ish edge, since they're discrete, non-continuous points. `ax.plot(...)` draws the predicted values as a continuous yellow line (because the fitted function can produce a prediction for any size) plus small yellow square markers at the five specific sizes. A loop using `iterrows()` — a pandas function that walks a DataFrame one row at a time — draws a short dashed red line between each actual and predicted point, visualizing that point's residual. `zorder` controls stacking order: background is `zorder` 1, connecting lines are `zorder` 2, and data-point markers are `zorder` 3, so the markers stay the most visible element. Cosmetic parameters like `color`, `edgecolor`, and `s` (marker size) carry no analytical meaning — they're comparable to choosing colors when creating a flowchart, and can be reused as fixed values across plots without memorizing them. For the five points, the visualized residuals are 0, 5, -2, -5, and 5 — exactly the values summed and squared to compute MSE.

```
y_actual = demo['actual_price'].values
y_pred = demo['predicted_price'].values
```



```

y_mean = np.mean(y_actual)
ss_residual = np.sum((y_actual - y_pred) ** 2)
ss_total = np.sum((y_actual - y_mean) ** 2)

r2_manual = 1 - (ss_residual / ss_total)
r2_sklearn = r2_score(y_actual, y_pred)

```

`.values` extracts the raw NumPy array from a pandas column, since the DataFrame itself is no longer needed once `y_actual` and `y_pred` are pulled out.

`SS_residual` is the model's own squared error (same numerator idea as MSE); `SS_total` is the baseline model's squared error against the mean.

The manual computation reproduces $R^2 \approx 0.9411$, and `r2_score`, imported earlier from `sklearn.metrics`, matches it.

Two extra experiments build intuition for R^2 's edge cases. Setting every predicted value equal to `y_mean` (the model always predicts the mean) recomputes R^2 as 0.00, confirming that a model indistinguishable from the baseline scores exactly zero. Substituting a deliberately random-looking predicted array (like 90, 40, 85, 55, 58 against very different actual values) recomputes R^2 as roughly -2.41. This confirms R^2 can fall below zero for a model worse than the naive baseline.

A second, larger practice dataset of 40 samples was generated for independent practice. It contains 40 house-size values from a normal distribution, clipped to fall between 400 and 2000 square feet.

Corresponding prices were generated using `price = 0.05 * size + 10 + noise`, where `noise` simulates real-world variation. The constants (0.05 as weight, 10 as bias) represent the underlying relationship a trained linear regression model should approximately recover. Learners are encouraged to fit a linear regression model on this dataset themselves and evaluate it, using only the data-generation cells as a starting point. Jupyter/Colab's autocomplete, triggered by typing a partial name and pressing Tab, helps discover available functions without memorizing exact syntax. What matters most is internalizing the underlying formulation and reasoning behind each metric, which is useful for interviews and conceptual understanding. The exact code syntax is secondary since it can always be looked up.

3. Key Takeaways

MSE and R^2 are complementary metrics for evaluating regression models — neither replaces the other, and both should be reported together.

MSE squares each residual, sums them, and divides by the number of points; it's preferred over MAE because squaring emphasizes large errors and produces a simpler derivative for optimization. RMSE is just the square root of MSE, and is the number to report to a non-technical customer since it stays in the original, interpretable unit scale.

R^2 compares a model's error against a naive baseline that always predicts the mean of the observed outputs. R^2 close to 1 means a near-perfect fit; R^2 of 0 means no improvement over the baseline; a negative R^2 means the model is worse than the baseline.

Both MSE and R^2 are sensitive to outliers, since both involve a squared-error term — outliers must be found and removed during data preparation, not left for the metric to handle.

Comparing MSE across two different datasets isn't valid, but comparing R^2 across datasets is defensible, since it's bounded and expresses improvement relative to each dataset's own baseline.

Testing on the same dataset is still generally preferred.

Mental model: Think of MSE as measuring how far off your model's predictions are in absolute terms. Think of R^2 as measuring how much smarter your model is than a lazy guesser who always predicts the average.